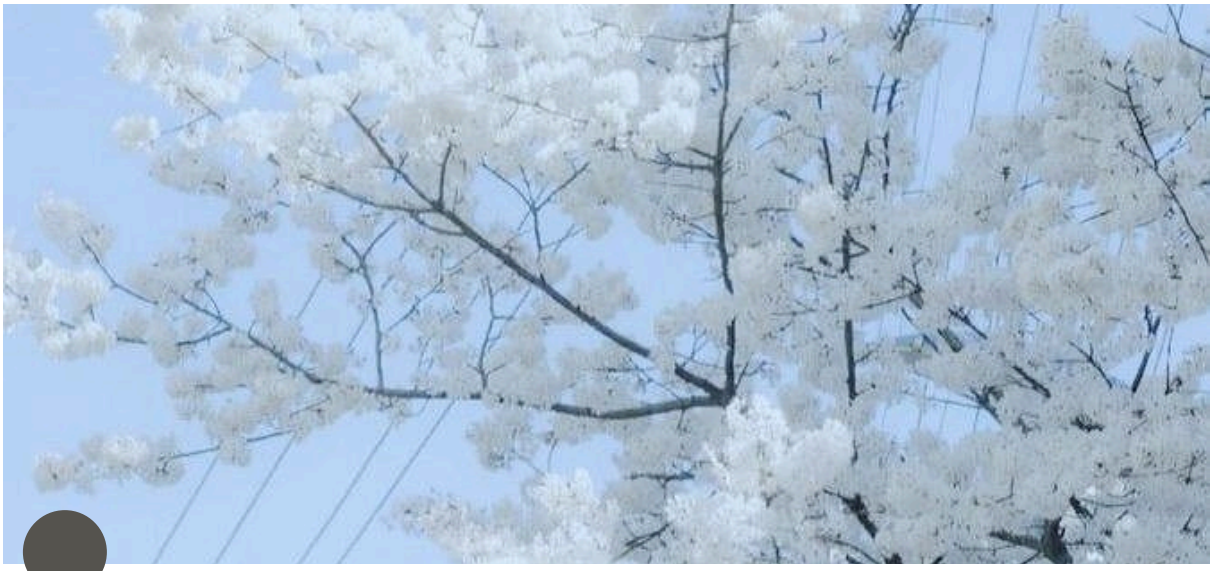


DBMS(Database Management System)

What is DBMS?

Data - Data is a collection of raw facts and information that can be stored, processed, and used to produce meaningful information.

Information - Information is processed and organized data that has meaningful context and helps us understand or make decisions.

```
Rahul, 85, 90, 80 //Data
Rahul's average marks = 85 //Information
```

Database - A database is an organized collection of related data that is stored electronically so that it can be easily accessed, managed, updated, and retrieved.

DBMS - DBMS is software used to create, store, manage, retrieve, and manipulate data in a database.

> Database - Where data is stored
>
> DBMS - Software that manage the data base

DBMS	File System
Reduces data redundancy and inconsistency	Data redundancy and inconsistency
Easy and efficient data access	Difficulty in accessing data
Provides data integration	Data isolation
Maintains data integrity using constraints	Integrity problems
Handles concurrent access	Concurrent access anomalies
Provides better security and access control	Security problems

Why to use DBMS?

1. Efficient Data Storage
2. easy Data retrieval
3. Reduces Data Redundancy
4. Maintains Data Consistency
5. Data Security
6. Data Integrity
7. Concurrent Access
8. Backup and Recovery

▼ DBMS Architecture

1. External Level (View Level)

- Highest level of abstraction.
- Shows only the data required by a particular user.
- Different users can have different views of the same database.

Example: A student can see their marks, while a teacher can see the marks of all students.

2. Conceptual Level (Logical Level)

- Describes the overall logical structure of the database.
- Defines tables, columns, relationships, and constraints.
- Hides physical storage details.

Example: A `Student` table containing `student_id`, `name`, and `marks`.

3. Internal Level (Physical Level)

- Lowest level of abstraction.
- Describes how data is physically stored.
- Includes storage files, indexes, and data organization.

Example: How student records are stored on disk and how indexes help retrieve them.

Data Base Instance -

The collection of information stored in the DB at a particular moment is called an instance of DB.

DB Schema(logical schema) -

- attribute of table
- consistent constants(Primary key)
- relationship

Data Models -

A data model is a collection of concepts used to describe the structure of a database, relationships between data, and constraints on the data.

Types of Data Models -

1. Hierarchical Data Model

- Organizes data in a tree-like structure.
- Each child typically has only one parent.

- Example: Organization hierarchy.

2. Network Data Model
   - Organizes data as a graph of interconnected records.
   - A record can have multiple parent records.
   - Supports many-to-many relationships.

3. Relational Data Model
   - Stores data in tables consisting of rows and columns.
   - Tables can be related using keys.
   - Example: MySQL, PostgreSQL, Oracle Database.

4. Entity-Relationship (ER) Model
   - Represents data using entities, attributes, and relationships.
   - Commonly used for database design.
   - Example: Student enrolls in Course.

5. Object-Oriented Data Model
   - Represents data as objects containing attributes and behavior.
   - Uses concepts such as classes, objects, and inheritance.

6. NoSQL Data Models
   - Used by many non-relational databases.
   - Common types include document, key-value, column-family, and graph models.
   - Examples: MongoDB (document), Redis (key-value).

Data Base Language -

Database languages are used to define, manipulate, retrieve, control, and manage data in a database.

1. DDL - Data Definition Language

Used to define or modify the structure of database objects such as tables.

Commands:

- `CREATE` – creates database objects.
- `ALTER` – modifies structure.
- `DROP` – deletes database objects.
- `TRUNCATE` – removes all rows from a table.

```
CREATE TABLE Student (
    id INT,
    name VARCHAR(50)
);
```

2. DML - Data Manipulation Language

Used to insert, update, and delete data in tables.

Commands:

- `INSERT`
- `UPDATE`
- `DELETE`

```
INSERT INTO Student
VALUES (101, 'Rahul');
```

3. DQL - Data Query Language

Used to retrieve data from a database.

Command:

- `SELECT`

```sql
SELECT * FROM Student;
```

4. DCL - Data Control Language

Used to control database access and permissions.

Commands:

- `GRANT` – gives permissions.
- `REVOKE` – removes permissions.

```sql
GRANT SELECT ON Student TO user1;
```

5. TCL - Transaction Control Language

Used to manage transactions in a database.

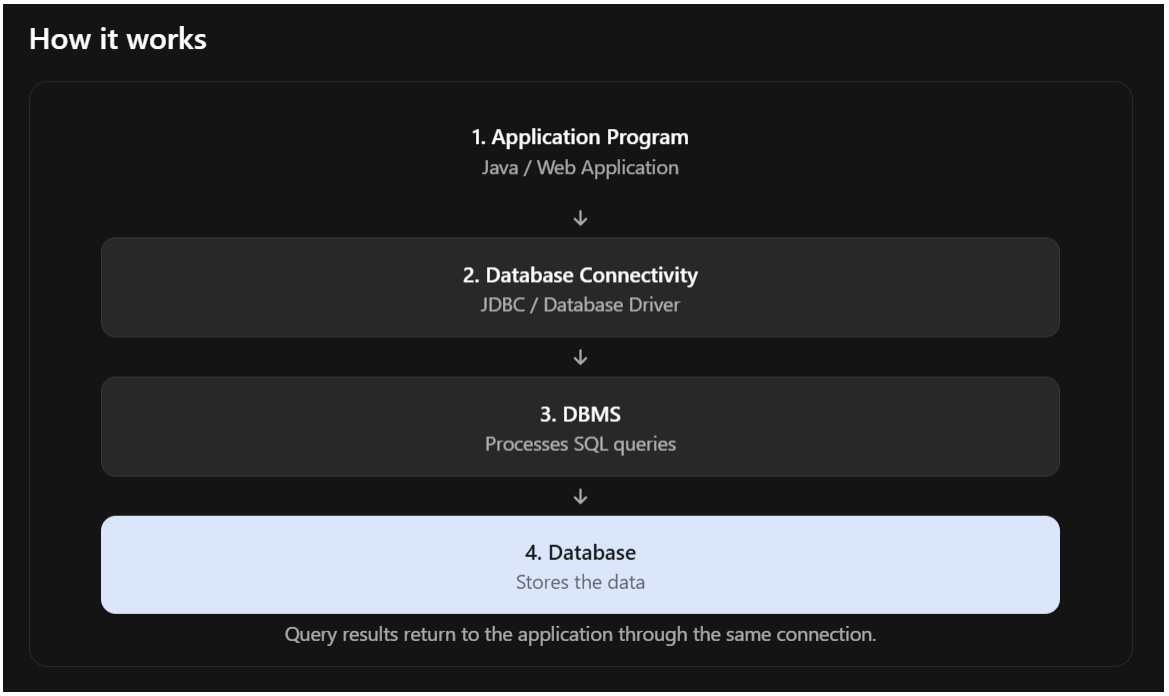
Commands:

- `COMMIT` – saves transaction changes.
- `ROLLBACK` – undoes uncommitted changes.
- `SAVEPOINT` – creates a point to roll back to.

```sql
COMMIT;
```

How is a Database Accessed from Application Programs?

An application program accesses a database through a database interface or connectivity driver that allows it to send queries and receive results.



Steps involved

1. The user performs an action in the application.
2. The application connects to the database using a connectivity interface or driver.
3. It sends an SQL query to the DBMS.
4. The DBMS processes the query and accesses or modifies the data.
5. The result is returned to the application.

Other ways to access a database

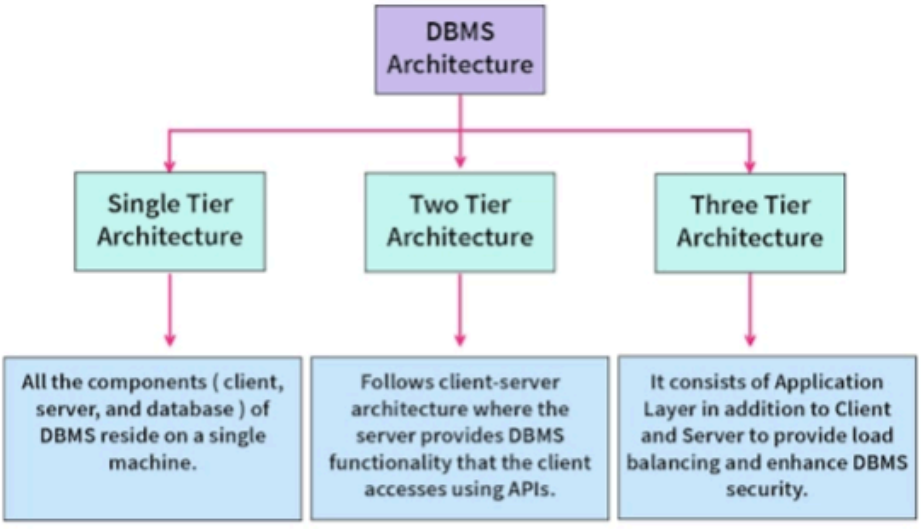
- JDBC: Direct database connectivity in Java.
- ORM frameworks: Hibernate / JPA map Java objects to database records.
- ODBC: A database connectivity standard used by different programming languages.

Data Base Administrator(DBA) -

A Database Administrator (DBA) is responsible for the security, maintenance, performance, backup, recovery, and overall management of a database system.

Responsibilities of a DBA

1. Database Design: Helps design the database structure.
2. Security Management: Controls user access and permissions.
3. Backup and Recovery: Backs up data and restores it after failures.
4. Performance Monitoring: Optimizes database performance and queries.
5. Data Integrity: Ensures data remains accurate and consistent.
6. Concurrency Management: Helps ensure multiple users can access the database safely.
7. Installation and Maintenance: Installs, configures, and updates database software.



▼ ER Model(Entity Relationship Model)

The ER Model is a conceptual data model used to represent the entities, attributes, and relationships in a database.

1. Components of the ER Model

A. Entity -

An entity is a real-world object that can be uniquely identified.

Examples: Student Rahul(ID 101),
Teacher Dr. Smith. Course CS101

- Entity type: A category of similar entities, such as Student.
- Entity instance: A specific entity, such as student Rahul with ID 101.
- Strong entity: An entity that has its own primary key.
- Weak entity: An entity that depends on another entity for identification and uses a partial key.

B. Entity Set -

It is a set of entities of the same type that share the same properties or attributes.

Example - The collection of all students currently enrolled in the university is the Student entity set.

ER diagram symbol: Rectangle.

Weak Entity Set symbol: Double Rectangle.

C. Attribute -

An attribute is a property that describes an entity/data.

Example: A Student has `student_id`, `name`, and `age`.

Attribute type	Meaning	Example
Simple	Cannot be divided further meaningfully	Age
Composite	Can be divided into smaller attributes	Address → City, State
Single-valued	Has one value per entity	Date of birth
Multi-valued	Can have multiple values	Phone numbers
Derived	Calculated from another attribute	Age from date of birth
Key attribute	Uniquely identifies an entity	Student ID

ER diagram symbols:

- Attribute → Oval
- Key attribute → Underlined oval attribute name
- Multivalued attribute → Double oval
- Derived attribute → Dashed oval

D. Relationship -

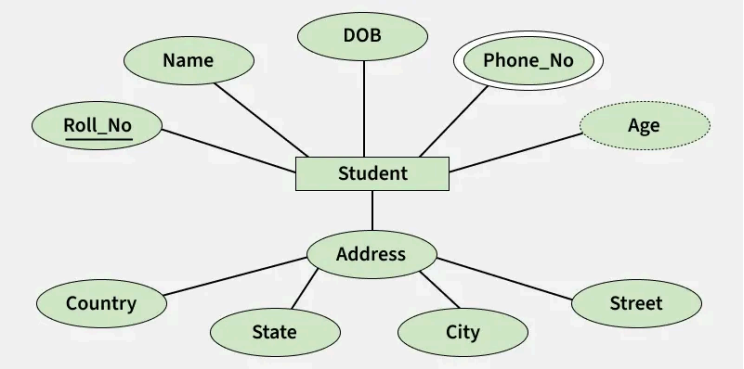
A relationship represents an association between two or more entities.

Example: A Student enrolls in a Course.

ER diagram symbol: Diamond.

ER Diagram -

It is a structural flowchart that acts as a blueprint for a database.



What does NULL specify in a database?

Null represents a missing, unknown, or unavailable value in a database. It is not the same as zero, an empty string, or a space.

Important points -

1. `NULL` means a value is missing, unknown, or unavailable.
2. `NULL` is different from `0`, `''` (empty string), and a space.
3. You cannot check for NULL using `= NULL`.
4. Use `IS NULL` or `IS NOT NULL` to check NULL values.

Example -

student_id	name	phor
101	Rahul	9876
102	Aman	NULL
103	Priya	

```sql
SELECT *
FROM Student
WHERE phone IS NULL;
```

Relationship -

A Relationship defines how two or more entity sets are interlinked or associated with each other in the real world.

Degree of Relationship -

The degree refers to the number of entity sets participating in a single relationship.

- **Unary (Recursive) Relationship (Degree 1):** An entity set relates to itself.

    - *Example:* An `Employee` **manages** another `Employee`.
- **Binary Relationship (Degree 2):** Exactly two entity sets participate. This is the most common type.
    - *Example:* `Student` **takes** `Exam`.
- **Ternary Relationship (Degree 3):** Three entity sets participate simultaneously.
    - *Example:* `Doctor` **prescribes** `Medicine` to a `Patient`.

Types of Relationships

Relationship	Meaning	Example
One-to-One (1:1)	One entity relates to one entity	Person – Passport
One-to-Many (1:M)	One entity relates to many entities	Department – Employees
Many-to-One (M:1)	Many entities relate to one entity	Employees – Department
Many-to-Many (M:N)	Many entities relate to many entities	Students – Courses

Cardinality Constraints

Cardinality specifies how many entities can participate in a relationship.

- 1:1 – One-to-one
- 1:N – One-to-many
- M:N – Many-to-many

Participation constraints:

- Total participation: Every entity must participate in the relationship.
- Partial participation: Participation is optional for some entities.

Advantages of the ER Model

1. Makes database design easier to understand.
2. Clearly represents entities and their relationships.
3. Helps identify attributes and keys.
4. Reduces design errors before implementation.
5. Can be converted into relational tables.

▼ Relational Model

The relational model is a data model that organizes data into tables (called relations) consisting of rows and columns. Relationships between tables are established using keys.

Example of a Relational Model

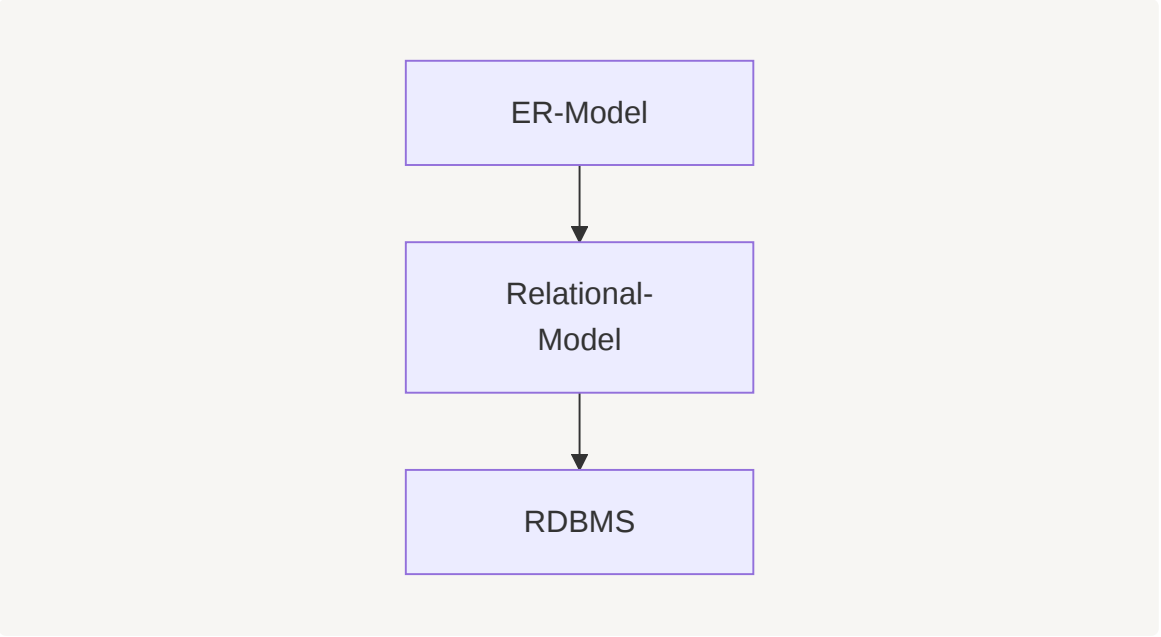
Student Table

student_id	name	age
101	Rahul	20
102	Aman	21
103	Priya	19

In the relational model:

- Relation: A table, e.g., `Student`.
- Tuple: A row in a table, e.g., `(101, Rahul, 20)`.
- Attribute: A column, e.g., `name`.
- Domain: The set of valid values allowed for an attribute, e.g., age as positive integers.
- Degree: Number of attributes (columns) in a relation. Here, 3
- Cardinality: Number of tuples (rows) in a relation. Here, 3

DB Design



What is RDBMS?

RDBMS (Relational Database Management System) is a type of DBMS that stores and manages data in tables consisting of rows and columns, with relationships between tables established using keys.

Important Properties of a Table in Relational Model

1. The name of relation is distinct among all other relation
2. The values have to be atomic. Can’t be broken down further
3. The name of each attribute/column must be unique
4. Each tuple must be unique in a table
5. The sequence of row and column has no significant

Features of RDBMS

1. Table-based structure: Stores data in rows and columns.
2. Relationships: Connects tables using primary and foreign keys.
3. Data integrity: Uses constraints to maintain valid data.
4. SQL support: Uses SQL to retrieve and manipulate data.

5. Normalization: Helps reduce unnecessary data redundancy.
6. Transaction management: Supports reliable transactions and ACID properties.
7. Security: Provides user permissions and access control.

Examples of RDBMS

- MySQL
- PostgreSQL
- Oracle Database
- Microsoft SQL Server
- SQLite

Keys in the Relational Model

A key is an attribute or a set of attributes used to uniquely identify a tuple (row) in a relation (table) or to establish relationships between tables.

1. Super Key -

   A set of one or more attributes that uniquely identifies each row in a table.

   Example: `Student(student_id, email, name)`

2. Candidate Key -

   A minimal super key — no attribute can be removed without losing uniqueness.

   Example: `{student_id}` , `{email}`

3. Primary Key -

   The candidate key selected to uniquely identify each row.

   Example: `student_id`

   Important points:

- Must be unique.
- Cannot contain `NULL`.
- Only one primary key constraint per table, though it may contain multiple columns.

4. Alternate Key -

   A candidate key that is not selected as the primary key.

   Example: If `student_id` is the primary key, `email` can be an alternate key.

5. Foreign Key -

   An attribute or set of attributes that references a primary key or another eligible unique key in the referenced table.

   Example: `course_id` in `Student` references `Course(course_id)`.

   Purpose: Maintains referential integrity between tables.

6. Composite Key -

   A key consisting of two or more attributes.

   Example: `Enrollment(student_id, course_id)`

   The combination of `student_id` and `course_id` uniquely identifies each enrollment.

7. Unique Key -

   In SQL, a `UNIQUE` constraint ensures that values—or combinations of values—are unique.

```
email VARCHAR(100) UNIQUE
```

8. Surrogate Key -

   A surrogate key is a system-generated identifier with no business meaning, used to uniquely identify records in a

database.

Usually generated automatically, such as using `AUTO_INCREMENT` in MySQL or an identity column.

```
CREATE TABLE Student (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50),
    email VARCHAR(100) UNIQUE
);
```

Integrity Constraints

CRUD Operation must ne done with some integrity policy so that DB is always consistent.

Introduce so that we do not accidentally corrupt the DB.

Types of Integrity Constraints

1. Domain Constraint -

Ensures that an attribute contains values from a valid domain (allowed data type, range, or set of values).
Example: Age must be a positive number

```
age INT CHECK (age > 0)
```

2. Key Constraint -

Ensures that key attributes uniquely identify records.

Example: Two students cannot have the same `student_id`.

```
student_id INT UNIQUE
```

3. Entity Integrity Constraint -

Ensures that a primary key cannot be NULL.

```
student_id INT PRIMARY KEY
```

4. Referential Integrity Constraint -

Ensures that a foreign key refers to an existing key in the referenced table, or is NULL when permitted.

```
CREATE TABLE Course (
    course_id INT PRIMARY KEY
);

CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    course_id INT,
    FOREIGN KEY (course_id)
        REFERENCES Course(course_id)
);
```

Other Common SQL Constraints

Constraint	Purpose
`NOT NULL`	Prevents NULL values
`UNIQUE`	Prevents duplicate values or combinations
`PRIMARY KEY`	Uniquely identifies each row and prohibits NULL
`FOREIGN KEY`	Enforces referential integrity
`CHECK`	Ensures values satisfy a condition
`DEFAULT`	Supplies a default value when one is not provided

▼ Transform ER Model to Relational Model

Transforming an ER Model into a Relational Model means converting entities, attributes, and relationships from an ER diagram into relational tables with appropriate primary keys and foreign keys.

Rules for Conversion

1. Strong Entity → Table

Create a separate table for each strong entity. Its simple attributes become columns, and its key attribute becomes the primary key.

ER Entity: Student (student_id, name, age)

Relational Table:

student_id (PK)	name	age
101	Rahul	20
102	Aman	21

2. Weak Entity → Table

Create a separate table for the weak entity. Include:

- Its own attributes
- The primary key of its owner entity as a foreign key
- A composite primary key, usually combining the owner's key with the weak entity's partial key

Example: Employee has Dependents.

`Dependent(employee_id, dependent_name, age)`

- PK: `(employee_id, dependent_name)`
- FK: `employee_id` references `Employee(employee_id)`

3. Simple and Composite Attributes

- Simple attribute: Store it directly as a column.
- Composite attribute: Store its component attributes as separate columns.

Example: `Address` becomes `street`, `city`, and `state`.

4. Multivalued Attribute → Separate Table

Create a separate table for the multivalued attribute and include the owner's primary key.

Example: A student can have multiple phone numbers.

`StudentPhone(student_id, phone_number)`

- PK: `(student_id, phone_number)`
- FK: `student_id` references `Student(student_id)`

5. Derived Attribute

Usually, do not store a derived attribute if it can be calculated from existing attributes.

Example: Calculate `age` from `date_of_birth` instead of storing both, where appropriate.

6. One-to-One (1:1) Relationship

Add a foreign key to one of the tables, preferably the side with total participation. Add a `UNIQUE` constraint to ensure the relationship remains one-to-one.

Example: Person – Passport

`Passport(passport_id PK, person_id FK UNIQUE)`

7. One-to-Many (1:N) Relationship

Add the primary key of the one-side table as a foreign key in the many-side table.

Example: One Department has many Employees.

8. Many-to-Many (M:N) Relationship

Create a separate junction table containing the primary keys of both participating entities as foreign keys. Together, they commonly form a composite primary key.

Example: Students enroll in many courses, and courses have many students.

`Enrollment(student_id, course_id)`

- PK: `(student_id, course_id)`
- Both columns are FKs referencing their respective tables.

9. N-ary Relationship

For a relationship involving three or more entities, create a separate table containing the primary keys of the participating entities as foreign keys, along with any relationship attributes.

How to Handle Generalization

Generalization is a bottom-up process in which two or more lower-level entities with common attributes are combined into a higher-level entity (superclass).

Method 1: Create a Table for the Superclass and Each Subclass -

Create three tables. The primary key of the superclass becomes the primary key and foreign key in each subclass.

- `Vehicle(vehicle_id PK, brand)`
- `Car(vehicle_id PK/FK, seats)`
- `Bike(vehicle_id PK/FK, engine_cc)`

Advantage: Avoids repeating common attributes and supports specialization well.

Disadvantage: Retrieving all vehicle details may require joins.

Method 2: Create Tables Only for the Subclasses -

Create a separate table for each subclass, including both common and subclass-specific attributes.

- `Car(vehicle_id PK, brand, seats)`
- `Bike(vehicle_id PK, brand, engine_cc)`

Advantage: No join is required to retrieve subclass details.

Disadvantage: Common attributes are repeated across tables. This works best when every superclass entity belongs to at

least one subclass.

Method 3: Create One Table for the Entire Hierarchy -

Create a single table containing superclass attributes, subclass-specific attributes, and a type discriminator.

`Vehicle(vehicle_id PK, brand, vehicle_type, seats, engine_cc)`

Example:

vehicle_id	brand	vehicle_type	seats	engine_cc
1	Toyota	Car	5	NULL
2	Honda	Bike	NULL	150

Advantage: Simple queries without joins.

Disadvantage: Many columns may contain `NULL`, and constraints are needed to ensure the right attributes are populated for each type.



The most common normalized approach is Method 1: a superclass table plus subclass tables, with each subclass primary key also referencing the superclass. The best mapping depends on whether the specialization is total or partial, disjoint or overlapping, and the application's querying needs.

Aggregation

Aggregation is handled by treating a relationship between entities as a higher-level unit so that it can participate in another relationship.

Example -

Suppose:

- An Employee works on a Project.
- A Manager monitors that particular work assignment.

Here, the manager monitors the relationship Works_On between Employee and Project, not just the employee or project separately.

How to handle aggregation when converting an ER diagram into relational tables -

1. Create an `Employee` table.
2. Create a `Project` table.
3. Represent `Works_On` using a separate table, such as `Works_On(employee_id, project_id)`.
4. Represent the monitoring relationship using another table, such as `Monitors(manager_id, employee_id, project_id)`.

```
CREATE TABLE Works_On (
    employee_id INT,
    project_id INT,
    PRIMARY KEY (employee_id, project_id)
);

CREATE TABLE Monitors (
    manager_id INT,
    employee_id INT,
    project_id INT,
    PRIMARY KEY (manager_id, employee_id, project_id),
    FOREIGN KEY (employee_id, project_id)
        REFERENCES Works_On(employee_id, project_id)
);
```

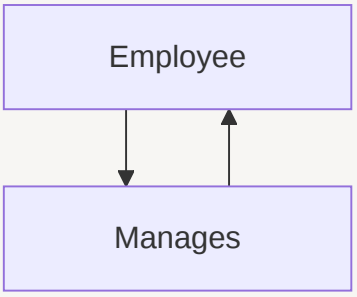
Aggregation is useful when a relationship itself needs to participate in another relationship. In relational mapping, represent the original relationship as a table and reference it from the higher-level relationship.

Unary Relationship

A unary relationship is a relationship in which an entity is related to another entity of the same entity set. It is also called a recursive relationship.

Example: Employee manages Employee -

An employee can manage another employee. Both entities belong to the same `Employee` entity set.



Types of unary relationships

- 1:1: One employee manages at most one other employee, and each employee is managed by at most one employee.
- 1:N: One employee manages many employees.
- M:N: Employees can collaborate with many other employees.

How to represent it in a relational table -

For a 1:N relationship, use a self-referencing foreign key.

```
CREATE TABLE Employee (
    employee_id INT PRIMARY KEY,
    name VARCHAR(50),
    manager_id INT,
    FOREIGN KEY (manager_id)
        REFERENCES Employee(employee_id)
);
```

▼ Normalization

Normalization is a step towards DB optimization.

Functional Dependency (FD)

A functional dependency is a relationship between attributes in a table where one attribute, or a group of attributes, uniquely determines another attribute.

It is represented as:

A ——> B

- A = Determinant (the attribute that determines another attribute)
- B = Dependent attribute

Example: Student table

Student_ID	Student_Name	Department
101	Rahul	CSE
102	Priya	IT
103	Aman	CSE

Here:
Student_ID ——> Student_Name

Student_ID ——> Department

The reverse is not necessarily true. Two students can have the same department, so `Department → Student_ID` does not hold.

Types of Functional Dependency

1. Trivial Functional Dependency -

A functional dependency X ——> Y is trivial if Y is a subset of X.

A–>A, B–>B is also a trivial FD

Example:
(Student_ID, Student_Name) ——> Student_Name

This is trivial because `Student_Name` is already part of the left-hand side.

2. Non-trivial Functional Dependency -

A functional dependency is non-trivial if B is not a subset of A.

[A intersection B is NULL].

Example:
Student_ID ——> Student_Name

Rules of FD – Armstrong's Axioms

Armstrong's axioms are a set of rules used to derive all functional dependencies logically implied by a given set of functional dependencies.

1. Reflexivity Rule -

If A is a set of attributes and B is a subset of A, then A –> B.

2. Augmentation Rule -

If B can be determined from A then adding an attribute to this functional dependency won't change anything.

If X–>Y holds, then AX–>BX holds too. 'X' being a set of attributes.

3. Transitivity -

If A determine B and B determine C, we can say that A determines C.

A–>B and B–>C then A–>C

Why Do We Need Normalization?

Normalization is needed to organize database tables, reduce unnecessary data redundancy, and prevent data anomalies.

Main reasons for normalization

1. Reduce data redundancy: Avoid storing the same information repeatedly.

2. Prevent update anomalies: Ensure a change to information does not require updating multiple unnecessary copies.
3. Prevent insertion anomalies: Allow new information to be added without requiring unrelated data.
4. Prevent deletion anomalies: Avoid accidentally losing important information when deleting a record.
5. Improve data consistency: Reduce the risk of conflicting values for the same information.
6. Maintain data integrity: Organize data and relationships so constraints can be applied properly.
7. Simplify database maintenance: Make the database easier to modify and manage.

What is redundant data?

Data redundancy means storing the same information unnecessarily in multiple places.

Consider this table:

Student_ID	Student_Name	Course_ID	Course_Name	Instructor
101	Rahul	C01	Java	Amit
101	Rahul	C02	SQL	Neha
102	Priya	C01	Java	Amit

Here, Rahul's name is repeated, and the Java course and instructor information are also repeated.

This repetition is an example of redundancy.

What happens if we have redundant data?

- Wasted storage: The same information is stored repeatedly.
- Data inconsistency: Different copies of the same information may have different values.

- Update problems: Multiple rows must be updated when the same information changes.
- Insertion problems: We may be unable to insert some information without unrelated information.
- Deletion problems: Deleting one record may accidentally remove other important information.

These problems lead to database anomalies.

Anomalies

Anomalies are problems that occur when inserting, updating, or deleting data in a poorly designed table.

1. Insertion Anomaly -

We cannot insert a new course unless a student is enrolled in it, if the table requires student and enrollment details together.

For example, we want to add `C03 – Python`, but no student has enrolled yet.

2. Update Anomaly -

Suppose the instructor for Java changes from Amit to Arjun.

We must update every row containing the Java course. If we update only one row, the database may contain conflicting instructor names.

3. Deletion Anomaly -

Suppose Priya is the only student enrolled in a course. Deleting her enrollment might also delete the only stored record of that course and its instructor.

What is normalization?

Normalization is the process of organizing database tables to reduce unnecessary data redundancy and prevent insertion, update, and deletion anomalies.

It involves decomposing poorly designed tables into smaller, well-structured tables while preserving relationships between

the data

Types of Normal Forms

Normal forms are levels or rules used to evaluate and improve database table design.

1NF

First Normal Form (1NF)

A table is in 1NF when every cell contains a single atomic value and there are no repeating groups.

Example: Store each phone number in a separate row rather than putting `9876, 8765` in one cell.

2NF

Second Normal Form (2NF)

A table must be in 1NF, and every non-prime attribute must depend on the whole of every candidate key, not just part of a composite key.

Main goal: Remove partial dependencies.

3NF

Third Normal Form (3NF)

A table must be in 2NF, and for every non-trivial FD (A –>X), either (A) is a super key or (X) is a prime attribute (part of at least one candidate key).

Main goal: Remove problematic transitive dependencies of non-prime attributes on keys.

BCNF

Boyce-Codd Normal Form (BCNF)

For every non-trivial functional dependency A–>B,A must be a super key.

BCNF is stricter than 3NF.

Advantages of normalization

1. Reduces redundancy: Avoids unnecessary duplication of data.
2. Improves consistency: Reduces the chance of conflicting copies of the same information.
3. Prevents anomalies: Helps avoid insertion, update, and deletion anomalies.
4. Improves data integrity: Maintains better relationships and constraints between tables.
5. Simplifies maintenance: Changes to a fact usually need to be made in one appropriate place.
6. Improves logical organization: Separates different entities and relationships into suitable tables.

▼ Transactions and ACID Properties

What is a transaction?

A transaction is a logical unit of work in a database that consists of one or more operations, such as reading, inserting, updating, or deleting data.

A transaction should complete successfully as a whole; if it fails, its incomplete changes should be undone.

ACID properties

ACID stands for:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

These properties help transactions remain reliable, even when errors occur or multiple transactions run at the same time.

1. Atomicity – All or Nothing

A transaction either completes all its operations or none of them.

If one operation fails, the database rolls back the transaction's changes.

Bank example: If ₽500 is deducted from A but cannot be added to B, the deduction is undone.

2. Consistency – Valid State to Valid State

A transaction must preserve database rules and constraints. It should take the database from one valid state to another valid state.

Bank example: The transfer must preserve the total money across the two accounts: ₽3,000 before and after.

Consistency also depends on rules such as primary keys, foreign keys, and application-defined constraints.

3. Isolation – Transactions Do Not Interfere Incorrectly

When transactions run concurrently, their execution should behave as though they ran in a valid serial order, according to the isolation level.

Example: If two users try to withdraw money from the same account simultaneously, isolation mechanisms help prevent incorrect results caused by overlapping operations.

Isolation levels differ: weaker levels may allow certain concurrency anomalies, while stronger levels prevent more of them.

4. Durability – Committed Changes Persist

Once a transaction is successfully committed, its changes should survive system failures, such as a crash or restart.

Example: After the bank confirms the transfer, the updated balances should remain saved even if the server crashes immediately afterward.

Transaction commands to remember

Command	Purpose
`START TRANSACTION` / `BEGIN`	Starts a transaction
`COMMIT`	Saves the transaction's changes
`ROLLBACK`	Undoes uncommitted changes
`SAVEPOINT`	Creates a point within a transaction to which you can roll back

▼

How to implement Atomicity and Durability in Transactions?

A DBMS uses a recovery mechanism to ensure that transactions remain reliable even if a system failure occurs.

The two main techniques used to support atomicity and durability are:

1. Shadow-copy scheme
2. Log-based recovery methods
    - Deferred database modifications
    - Immediate database modifications

Recovery Mechanism

Definition: A recovery mechanism is a component of a DBMS that restores the database to a correct, consistent state after a

failure.

It helps ensure:

- Atomicity: Incomplete transactions do not leave partial changes behind.
- Durability: Committed transactions are not lost after a failure.

Shadow-Copy Scheme

Definition: The shadow-copy scheme maintains an unchanged copy of the database pages and writes modifications to a separate copy until the transaction commits.

How it works -

1. The DBMS keeps the original database copy, called the shadow copy.
2. Changes are written to new database pages rather than overwriting the original pages.
3. If the transaction succeeds, the DBMS atomically switches the database pointer to the new copy.
4. If the transaction fails, the new copy is discarded, and the original copy remains available.

How it supports ACID

- Atomicity: Failed transactions leave the original database unchanged.
- Durability: After commit, the new database copy and its metadata must be safely persisted.

Limitation: Copying or managing pages can be expensive, and concurrent transactions and storage reclamation require additional care.

Log-Based Recovery Methods

Definition: Log-based recovery uses a log file to record transaction operations and database changes so the DBMS can recover after a failure.

A log is stored on persistent storage and may contain records such as:

- Transaction started: `<T1 START>`
- Data changed: `<T1, A, old_value, new_value>`
- Transaction committed: `<T1 COMMIT>`

For example, if transaction `T1` changes account A's balance from ₽2,000 to ₽1,500, the log may record both the old and new values.

Write-Ahead Logging (WAL)

The main rule is:

The relevant log record must be written to stable storage before the corresponding modified database page is written to disk.

The commit record must also be safely persisted before the DBMS acknowledges a successful commit.

This allows the DBMS to undo or redo changes during recovery.

There are two main log-based approaches.

Deferred Database Modification

Definition: In deferred modification, the DBMS delays applying a transaction's database changes until the transaction commits.

How it works -

1. The transaction starts.
2. The DBMS records the intended changes in the log.

3. The actual database pages are not updated by that transaction before commit.
4. If the transaction commits, the changes are applied during execution or recovery.
5. If it fails before commit, its changes are discarded.

Recovery rule -

- Committed transaction: REDO its changes if they have not reached the database.
- Uncommitted transaction: No UNDO is needed because its changes were not applied to the database.

Immediate Database Modification

Definition: In immediate modification, the DBMS may write a transaction's changes to database pages before the transaction commits.

Because uncommitted changes may reach disk, the DBMS must be able to undo them if the transaction fails.

How it works -

1. The transaction starts.
2. The DBMS records the old and new values in the log.
3. Database pages may be updated before commit.
4. If the transaction commits, its changes must be preserved.
5. If it fails, the DBMS uses the log to undo its incomplete changes.

Recovery rule -

- Committed transaction: REDO its changes if necessary.
- Uncommitted transaction: UNDO its changes if they reached the database.

▼ Why Indexing is important in Database Systems

What is Indexing?

Indexing is a technique used in DBMS to speed up data retrieval by creating a separate data structure that helps locate records without scanning the entire table.

It works like the index at the back of a textbook: instead of reading every page, you look up a topic and go directly to the relevant page.

Example -

Suppose we have a `Student` table with 10,000 records and want to find the student whose ID is `1050`.

Without an index, the DBMS may scan many rows to find the record. With an index on `Student_ID`, it can locate the record more efficiently.

```
CREATE INDEX idx_student_id
ON Student(Student_ID);
```

What is a Search Key?

A search key is an attribute or a set of attributes used to search for records in a database index.

Example: If we create an index on `Student_ID`, then `Student_ID` is the search key.

Student_ID	Name	Department
101	Rahul	CSE
102	Priya	IT
103	Aman	CSE

Here, `Student_ID` can be the search key. A search key does not have to be unique; an index can also be built on `Department`, where multiple students may have the same value.

💡 A search key is not necessarily the same as a primary key.

What is a Data Reference?

A data reference, often called a pointer or record pointer, is information stored in an index entry that helps the DBMS locate the corresponding record or data page.

An index entry conceptually contains:

- Search-key value: The value being searched.
- Pointer/reference: Where the matching record or page can be found.

Is an Index File Always Sorted?

No, an index file is not always sorted. It depends on the indexing method.

- Ordered index: Index entries are maintained in sorted order of the search key.
- Hash index: Entries are organized using a hash function, not necessarily in sorted order.

Indexing Methods

There are several ways to classify database indexes.

1. Primary Index

An index built on the ordering primary key of a file, in the traditional DBMS textbook classification.

- The data file is physically ordered by that key.
- The key values are unique.

- A sparse index may be used.

2. Clustering Index

An index built on the attribute by which the data file is physically ordered, when that attribute is not necessarily unique.

Example: Student records are stored grouped by `Department`.

Several students can belong to CSE, so the clustering field may contain duplicate values.

3. Secondary Index (Non-clustering Index)

An index built on a field that does not determine the physical ordering of the data file.

Example: Records are physically ordered by `Student_ID`, but an additional index is created on `Department`.

The department index helps find students by department without rearranging the underlying records.

4. Dense Index

A dense index has an index entry for every search-key value or every record, depending on the design.

- Provides direct lookup through the index.
- Requires more index storage.

5. Sparse Index

A sparse index stores entries for only some search-key values, often one entry per data block.

- Requires less storage.
- Works when the underlying data is ordered appropriately.

6. Multilevel Index

An index is created over another index to reduce the number of index entries that must be searched. B-trees and B+ trees

support efficient multilevel indexing.

7. Hash Index

A hash function maps a search-key value to a bucket containing references to matching records.

- Good for equality lookups such as `WHERE Student_ID = 101`.
- Not generally suitable for range queries such as `WHERE Student_ID BETWEEN 100 AND 200`.

Secondary Index (Non-clustering Index)

A secondary index is an index created on an attribute that does not determine the physical ordering of the data records.

Example -

Suppose the `Student` table is physically ordered by `Student_ID`.

Student_ID	Name	Department
101	Rahul	CSE
102	Priya	IT
103	Aman	CSE
104	Neha	IT

Now create an index on `Department`.

Department	Data references
CSE	Records 101, 103
IT	Records 102, 104

When searching for all CSE students, the DBMS can use the secondary index to locate their records.

Important points:

- The index order can differ from the physical order of the data records.
- Multiple records may share the same search-key value.

- In the traditional textbook classification, a secondary index is generally dense.

Advantages of Indexing

1. Faster data retrieval: Reduces the need to scan the entire table.
2. Efficient searching: Helps locate records using indexed columns.
3. Faster sorting and range queries: Ordered indexes can help with `ORDER BY`, range conditions, and some grouping operations.
4. Efficient joins: Indexes on join columns can speed up some join operations.
5. Supports uniqueness: A unique index can enforce uniqueness where supported.

Limitations of Indexing

1. Extra storage: Indexes consume additional disk space.
2. Slower inserts: The DBMS must maintain indexes when new rows are added.
3. Slower updates: Updating an indexed column may require index changes.
4. Slower deletes: Removing records may require updating index structures.
5. Maintenance overhead: Indexes need to be maintained and sometimes rebuilt or reorganized.
6. Not always beneficial: Indexes may provide little benefit for small tables or queries that return a large percentage of the table.

▼ NoSQL Database

What is a NoSQL Database?

NoSQL is a category of databases designed to store and manage data using models other than, or in addition to, the traditional relational table model.

NoSQL is commonly expanded as “Not Only SQL.” These databases often support flexible schemas, distributed storage, and horizontal scaling.

Unlike traditional relational databases, NoSQL databases may store data as documents, key-value pairs, graphs, or wide-column records.

Examples:

- MongoDB
- Redis
- Apache Cassandra
- Neo4j

Example of a document in MongoDB:

```
{
  "student_id": 101,
  "name": "Rahul",
  "skills": ["Java", "SQL"],
  "department": "CSE"
}
```

History Behind NoSQL

NoSQL did not suddenly replace SQL. It became more prominent as internet applications grew and needed to handle large amounts of distributed data.

- 1970s: The relational model and relational databases became influential; SQL-based systems developed and matured.
- 1990s-early 2000s: Web applications began generating increasingly large volumes of data.
- 2000s: Companies such as Google and Amazon published work on distributed data systems, including Bigtable and Dynamo,

addressing scalability and availability challenges.

- Late 2000s onward: The term NoSQL gained popularity, and databases such as MongoDB, Cassandra, and Redis became widely known.
- Today: SQL and NoSQL databases coexist, each serving different requirements.

Main reason for the growth of NoSQL: The need to manage large, rapidly changing, and distributed workloads efficiently.

Advantages of NoSQL Databases

1. Flexible schema: Records can often have different fields or structures.
2. Horizontal scalability: Many NoSQL systems can scale by distributing data across multiple machines.
3. High throughput: Some are optimized for large volumes of reads and writes.
4. Suitable for varied data: Can represent documents, relationships, key-value data, and other structures.
5. Distributed architecture: Many support replication and partitioning across servers.
6. Rapid development: Flexible data models can make it easier to adapt to changing application requirements.

When Should We Use NoSQL?

Use a NoSQL database when its data model and scaling characteristics fit the application's requirements.

Common use cases include:

- Social media: User profiles, feeds, comments, and activity data.
- Real-time applications: Caching, session storage, live counters, and leaderboards.

- IoT applications: Large volumes of sensor readings.
- Product catalogs: Products with different attributes and nested details.
- Content management: Documents, articles, and flexible content structures.
- Graph applications: Social connections, recommendations, and network analysis.
- Distributed applications: Workloads that need data distributed across multiple servers.

Example: A product catalog may contain laptops with RAM and processor details, while clothing products contain size and color details. A document database can represent these different structures naturally.

NoSQL Database Misconceptions

Misconception	Reality
NoSQL means SQL is not allowed.	It means “Not Only SQL”; some systems support SQL-like query languages.
NoSQL is always faster than SQL.	Performance depends on workload, schema, queries, and configuration.
NoSQL has no schema.	It may have a flexible schema, but data structure and validation rules still matter.
NoSQL cannot support transactions.	Many NoSQL databases support transactions, though capabilities differ.
NoSQL is always more scalable.	Many are designed for horizontal scaling, but relational databases can scale too.
NoSQL is always the best choice for big data.	The right choice depends on data relationships, consistency, workload, and operational needs.
NoSQL completely replaces SQL.	Both approaches are useful and often coexist in the same application.

Types of NoSQL Data Models

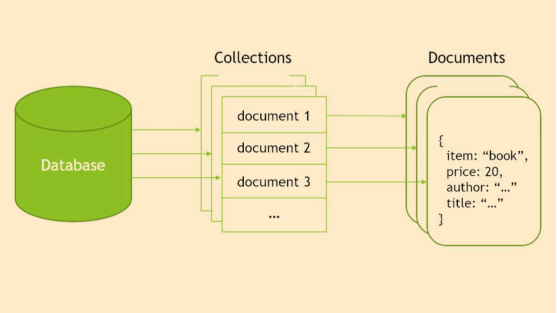
There are four major types.

1. Document Database

Stores data as documents, usually in JSON-like or binary JSON formats.

Example: MongoDB

Use cases: Product catalogs, user profiles, content management.



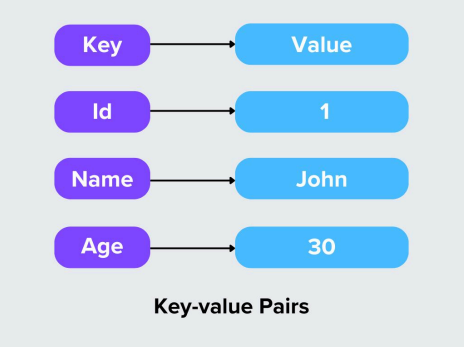
2. Key-Value Database

Stores data as a key and its associated value.

Example: Redis

```
"user:101" → "Rahul"
```

Use cases: Caching, sessions, counters.

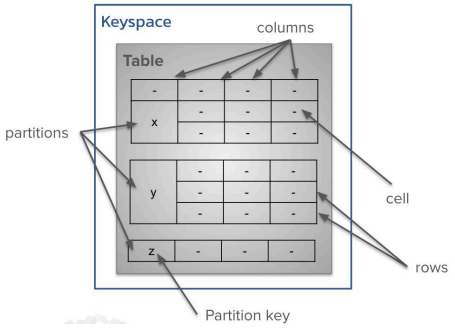


3. Wide-Column Database

Organizes data into rows and flexible sets of columns, with storage and distribution designed for large-scale workloads.

Example: Apache Cassandra

Use cases: Large-scale event data, telemetry, distributed write-heavy workloads.

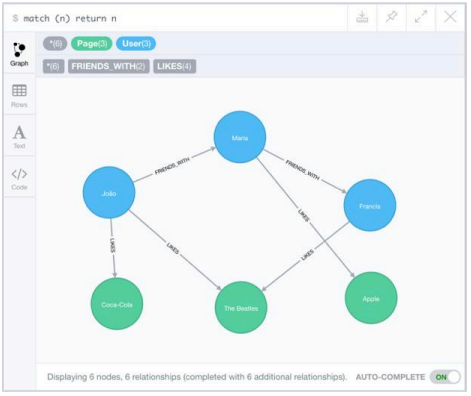


4. Graph Database

Stores data as nodes and relationships (edges), often with properties attached to them.

Example: Neo4j

Use cases: Social networks, recommendation engines, fraud detection.



Disadvantages of NoSQL Databases

1. Limited standardization: Different databases use different data models, query languages, and APIs.
2. Complex relationships: Some models make relational joins and complex relationships less convenient.
3. Data duplication: Denormalized designs may duplicate data, requiring careful updates.

4. Consistency trade-offs: Some distributed systems offer eventual consistency for particular operations or configurations.
5. Transaction differences: Transaction support and guarantees vary between products.
6. Migration challenges: Switching between NoSQL products may require significant data-model and application changes.
7. Learning curve: Choosing suitable partition keys, data structures, and replication settings can be challenging.

SQL vs NoSQL

Feature	SQL databases	NoSQL databases
Data model	Primarily relational tables	Document, key-value, wide-column, graph, etc.
Schema	Usually predefined and structured	Often flexible; varies by database
Query language	SQL is widely standardized	Varies by database
Relationships	Foreign keys and joins are common	May use references, embedded data, graph links, or other approaches
Transactions	Strong ACID support is common	Support and guarantees vary; many support ACID transactions
Scaling	Vertical and horizontal scaling are possible	Many are designed for horizontal distribution
Data consistency	Strong consistency is commonly supported	Depends on the database and configuration
Best suited for	Structured data, complex joins, financial transactions	Workloads that benefit from specific non-relational models or distributed designs
Examples	MySQL, PostgreSQL, SQL Server	MongoDB, Redis, Cassandra, Neo4j

▼ Types of Databases

Types of Databases –

A database is an organized collection of data that can be stored, accessed, and managed efficiently. Databases are classified based on how they organize and represent data.

The main types of databases are:

1. Relational Database
2. Object-Oriented Database
3. NoSQL Database
4. Hierarchical Database
5. Network Database

1. Relational Database (RDBMS)

A relational database stores data in the form of tables consisting of rows and columns. Tables can be related to each other using keys such as primary keys and foreign keys.

Example: Student Database

Student_ID (PK)	Name	Department_ID (FK)
101	Rahul	1
102	Amit	2
103	Priya	1

- Row (Tuple): Represents a single record.
- Column (Attribute): Represents a property of the data.
- Primary Key: Uniquely identifies each record.
- Foreign Key: Creates a relationship between tables.
- Query Language: SQL.

Examples: MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server.

Advantages -

- Easy to organize and query structured data.
- Supports relationships using keys and joins.
- Supports transactions and ACID properties.
- Constraints help maintain data integrity.

Disadvantages -

- A rigid schema can make frequent structural changes difficult.
- Complex joins may become expensive on large datasets.
- Scaling across multiple servers can require additional design and infrastructure.

Use cases: Banking, student management, employee records, and e-commerce transactions.

2. Object-Oriented Database (OODBMS)

An object-oriented database stores data as objects, similar to objects in object-oriented programming languages such as Java.

An object can contain attributes (data) and methods (behavior). Object-oriented databases can support concepts such as classes, inheritance, and object identity.

Example: Student Object

```
class Student {
    int id;
    String name;

    void display() {
        System.out.println(name);
    }
}
```

An object-oriented database can store a `Student` object directly, rather than requiring the application to map it into relational tables.

Examples: ObjectDB, db4o (legacy/discontinued).

Advantages -

- Works naturally with object-oriented applications.
- Supports complex objects and object relationships.
- Can reduce object-relational mapping effort.
- Useful for storing multimedia, engineering, and scientific objects.

Disadvantages -

- Less widely adopted than relational databases.
- Fewer standardized query tools and skills available.
- Data sharing and reporting can be less convenient when relational tools are expected.
- Can create tighter coupling between the application's object model and data storage.

Use cases: CAD/CAM systems, scientific applications, simulations, and complex object-based data.

3. NoSQL Database

NoSQL stands for Not Only SQL. NoSQL databases use data models other than, or alongside, the traditional relational table model. They are often designed for flexible data structures and distributed workloads.

Types of NoSQL databases

Type	Data storage	Example
Document	JSON-like documents	MongoDB
Key-Value	Key and associated value	Redis
Wide-Column	Column families	Cassandra
Graph	Nodes and relationships	Neo4j

Advantages -

- Flexible schema for changing data structures.

- Many systems support horizontal scaling across servers.
- Useful for large volumes of semi-structured or unstructured data.
- Different database models suit different workloads.

Disadvantages -

- Query languages and tools vary between systems.
- Complex relationships may require extra work in some models.
- Transaction and consistency guarantees differ by database.
- Data duplication can occur in some designs.

Use cases: Caching, social media feeds, product catalogs, IoT data, and recommendation systems.

Important: NoSQL is not always faster than SQL, and not every NoSQL database sacrifices transactions or consistency.

4. Hierarchical Database

A hierarchical database organizes data in a tree-like structure, where records have parent-child relationships.

Typically, each child has only one parent, so the model naturally represents one-to-many (1:M) relationships.

Advantages -

- Simple and easy to understand for tree-shaped data.
- Efficient when accessing data along predefined parent-child paths.
- Parent-child relationships are naturally represented.

Disadvantages -

- Does not naturally support many-to-many relationships.
- A child typically cannot have multiple parents.
- Structural changes can be difficult.
- Access paths are relatively rigid.

Use cases: Organization charts, file systems, and applications with fixed tree-like structures.

5. Network Database

A network database organizes records as a graph of interconnected records. Unlike the hierarchical model, a child record can have multiple parent records.

It can represent both one-to-many and many-to-many relationships.

Advantages -

- Supports complex relationships between records.
- Naturally represents many-to-many relationships.
- Can provide efficient navigation through predefined links.
- A record can participate in multiple relationships.

Disadvantages -

- Structure and relationships can be difficult to design and maintain.
- Navigation often depends on predefined access paths.
- Less flexible for ad hoc queries than modern relational systems.
- Requires developers to understand the database's record and link structure.

Use cases: Legacy business systems and applications with complex, predefined record relationships.